

House Price Prediction Using Machine Learning

A Complete Machine Learning Regression Project

Field	Details
Dataset	house_price.csv
Rows	50,000
Columns	19
Target Variable	price
Final Model	Linear Regression
R ² Score	0.9981
RMSE	19,941

B.Tech Computer Science — Academic Project

Technologies: Python | pandas | scikit-learn | matplotlib | seaborn

1. Project Overview

This project presents a complete end-to-end Machine Learning regression system designed to predict residential house prices. The system is built using Python and the scikit-learn library, following best practices including proper train-test splitting, pipeline-based preprocessing to prevent data leakage, and rigorous model evaluation.

The project covers every stage of the data science lifecycle: data understanding, preprocessing, exploratory data analysis (EDA), model training, evaluation, feature importance analysis, and deployment-ready model saving. Four regression algorithms are trained and compared, and the best model is selected based on objective metrics.

2. Problem Statement

House prices depend on a large number of factors including physical characteristics (area, number of rooms), location, age, neighbourhood amenities, crime rate, and income levels. Manual estimation of house prices is subjective and inconsistent.

The goal of this project is to build a data-driven machine learning model that can accurately predict the price of a residential property given its features, providing an objective, reproducible, and scalable pricing solution.

3. Objectives

- Load and inspect the house_price.csv dataset.
- Perform comprehensive Exploratory Data Analysis with visualisations.
- Preprocess data using pipelines to prevent data leakage.
- Train and compare Linear Regression, Decision Tree, Random Forest, and Gradient Boosting models.
- Evaluate all models using MAE, MSE, RMSE, and R².
- Select the best model based on evaluation metrics.
- Analyse feature importance to understand price drivers.
- Save the final model and demonstrate sample predictions.

4. Dataset Description

Property	Value
File name	house_price.csv
Total rows	50,000
Total columns	19
Target variable	price (continuous, float64)
Missing values	0 (none)
Duplicate rows	0 (none)
Numerical columns	16
Categorical columns	2 (location, income_level)

Price Statistics:

Statistic	Value
Minimum	90,900.69
25th Percentile	635,737.50
Median	1,031,763
Mean	1,030,261
75th Percentile	1,423,102
Maximum	1,996,474
Std Dev	458,020

5. Feature Description

Feature	Type	Description
area	Numerical	Property area in sq ft (500–4999)
bedrooms	Numerical	Number of bedrooms (1–6)
bathrooms	Numerical	Number of bathrooms (1–4)
floors	Numerical	Number of floors (1–3)
age	Numerical	Age of property in years (0–30)
distance	Numerical	Distance from city centre in km
garage	Binary	Has garage: 1 = Yes, 0 = No
parking	Binary	Has parking: 1 = Yes, 0 = No
garden	Binary	Has garden: 1 = Yes, 0 = No
security	Binary	Has security: 1 = Yes, 0 = No
school_nearby	Binary	School nearby: 1 = Yes, 0 = No
hospital_nearby	Binary	Hospital nearby: 1 = Yes, 0 = No
shopping_mall_nearby	Binary	Mall nearby: 1 = Yes, 0 = No
public_transport	Binary	Public transport nearby: 1 = Yes, 0 = No
crime_rate	Numerical	Crime rate in the area (float)
population_density	Numerical	Population density (100–9999)
location	Categorical	low / medium / premium
income_level	Categorical	low / mid / high
price	TARGET	House price (continuous float)

6. Data Preprocessing

6.1 Missing Values

All 19 columns were inspected for missing values. No missing values were found in any column. The dataset is complete and clean.

6.2 Duplicate Rows

Duplicate row analysis was performed. Zero duplicate rows were found in the 50,000-row dataset.

6.3 Numerical Feature Preprocessing

Numerical features are processed through a two-step pipeline:

- `SimpleImputer (strategy=median)`: Provides robustness against future missing values.
- `StandardScaler`: Standardises each feature to zero mean and unit variance. This ensures that features with large scales (e.g., area, population_density) do not dominate models that are sensitive to feature scale.

6.4 Categorical Feature Preprocessing

The two categorical columns (location, income_level) are encoded using:

- `SimpleImputer (strategy=most_frequent)`: Handles any unexpected missing values.
- `OneHotEncoder (handle_unknown=ignore)`: Creates binary indicator columns for each category. This avoids ordinal assumptions and is the correct encoding for nominal categories.

After encoding, the categorical columns expand into 6 binary features (3 location categories + 3 income_level categories).

6.5 Pipeline and Data Leakage Prevention

All preprocessing is wrapped in a `sklearn Pipeline + ColumnTransformer`. The train-test split is performed BEFORE fitting any preprocessing transformer. This prevents data leakage — statistics (mean, std, encoder categories) are computed only from training data and applied to test data.

7. Exploratory Data Analysis

EDA was performed to understand feature distributions, relationships with the target variable, and the overall structure of the data. Key findings are described below.

7.1 Target Variable Distribution

The price distribution is approximately bell-shaped (roughly normal) with a mean of 1,030,261 and a standard deviation of 458,020. Prices range from about 90,900 to 1,996,474.

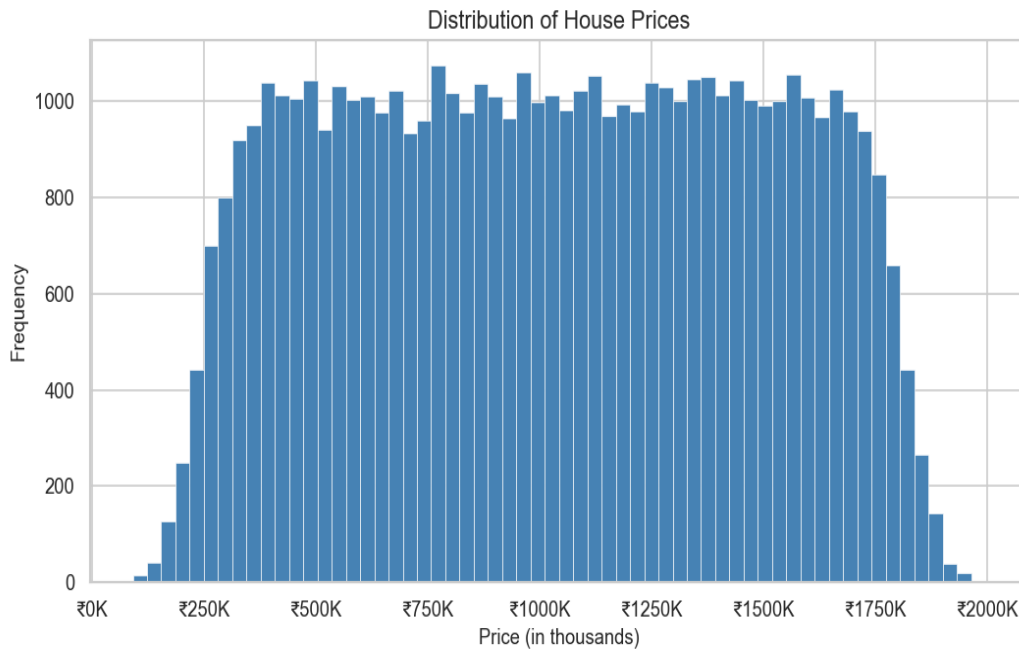


Figure 1: Distribution of House Prices

7.2 Area vs Price

The scatter plot of area vs price reveals a very strong positive linear relationship (Pearson correlation = 0.991). This means area is the single most important predictor of house price in this dataset.



Figure 2: Area vs House Price

7.3 Correlation Heatmap

The correlation heatmap confirms that area dominates all other features. Most other numerical features show very weak correlations with price ($|r| < 0.06$), suggesting that the non-area features contribute relatively less on their own but still provide predictive value in combination.

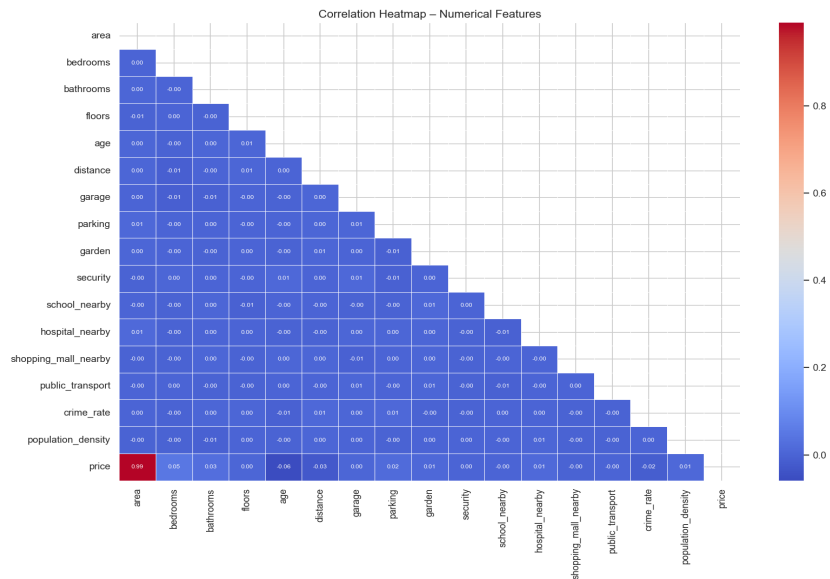


Figure 3: Correlation Heatmap

7.4 Location and Income Level vs Price

Premium locations have the highest average prices, followed by medium and low locations. High income level areas also command higher average prices compared to mid and low income level areas.

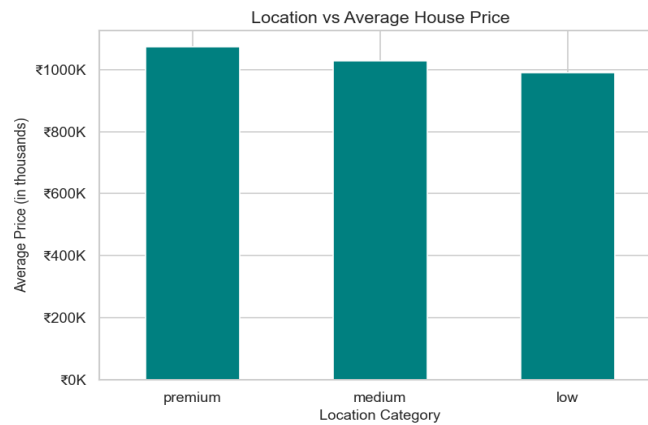


Figure 4: Location vs Average Price

8. Machine Learning Methodology

The dataset is split into 80% training (40,000 rows) and 20% testing (10,000 rows) using a fixed random_state=42 for reproducibility. Four regression models are trained using sklearn pipelines that combine preprocessing and the model estimator. Each pipeline is fitted only on training data and evaluated on held-out test data.

8.1 Train-Test Split

Split	Samples	Percentage
Training	40,000	80%
Testing	10,000	20%

9. Models Used

Model	Key Hyperparameters	Type
Linear Regression	Default (no tuning needed)	Linear
Decision Tree	random_state=42	Tree-based
Random Forest	n_estimators=100, random_state=42	Ensemble
Gradient Boosting	n_estimators=200, lr=0.1, depth=5	Ensemble Boosting

Linear Regression assumes a linear relationship between features and the target. Decision Tree creates recursive binary splits. Random Forest builds 100 trees and averages their predictions. Gradient Boosting trains trees sequentially, each correcting the errors of the previous one.

10. Model Evaluation

Metrics Explained

Metric	Formula	Interpretation
MAE	$\text{mean}(y - y_{\text{pred}})$	Average absolute error in price units; easy to interpret
MSE	$\text{mean}((y - y_{\text{pred}})^2)$	Penalises large errors more than MAE
RMSE	$\sqrt{\text{MSE}}$	Same unit as price; most interpretable error metric
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Fraction of variance explained; 1.0 = perfect fit

Evaluation Results

Model	MAE	MSE	RMSE	R ²
Linear Regression	15,954	397,637,516	19,941	0.9981
Decision Tree	34,454	1,891,657,258	43,493	0.9910
Random Forest	21,519	730,805,400	27,033	0.9965
Gradient Boosting	17,046	453,902,100	21,305	0.9978



Figure 5: Model RMSE and R² Comparison

11. Model Comparison

Linear Regression achieves the best performance across all four metrics: the lowest MAE (15,954), lowest RMSE (19,941), and the highest R² (0.9981). This is consistent with the extremely strong linear relationship between area and price ($r = 0.991$) discovered during EDA.

Gradient Boosting ranks second (RMSE = 21,305, R² = 0.9978), followed by Random Forest (RMSE = 27,033, R² = 0.9965). The Decision Tree shows the weakest performance (RMSE = 43,493, R² = 0.9910) due to high variance from overfitting individual training patterns.

12. Final Model Selection

Final Model: Linear Regression

Justification:

- Lowest MAE (15,954) — smallest average prediction error.
- Lowest RMSE (19,941) — best typical error in the unit of price.
- Highest R² (0.9981) — explains 99.81% of variance in house price.
- Simplest and most interpretable model — aligns with the dominant linear area-price relationship in the data.
- Less susceptible to overfitting on unseen data compared to deep tree-based models.

The saved model file includes the full preprocessing pipeline (imputer + scaler + OHE), ensuring it can be used for prediction on raw, unseen input data directly.

13. Feature Importance

Feature importance for Linear Regression is measured by the absolute value of the standardised coefficients — a larger value means that feature has a stronger influence on the predicted price.

Rank	Feature	Importance Score
1	area	454,027
2	location_low	40,165
3	location_premium	40,091
4	age	28,938
5	income_level_high	26,631
6	income_level_low	23,446
7	bedrooms	20,502
8	distance	14,988
9	bathrooms	10,161
10	crime_rate	8,794

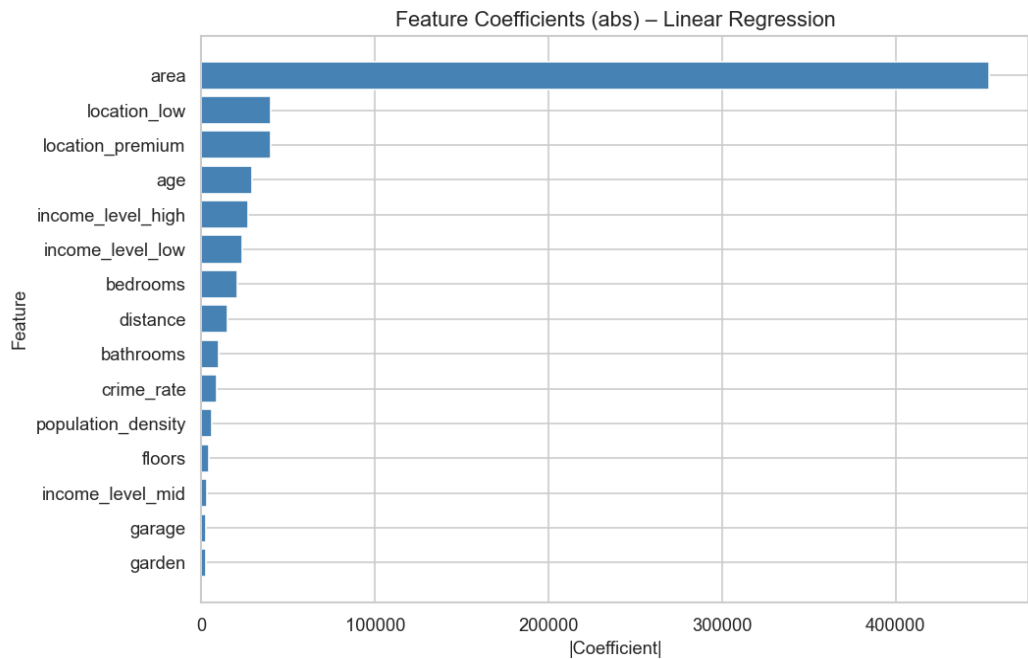


Figure 6: Feature Importance (Top 15 Features)

Key finding: area is overwhelmingly the most important feature — its coefficient is more than 10× larger than the next feature. Location category and house age are the second and third most influential factors.

14. Actual vs Predicted Analysis

The Actual vs Predicted scatter plot shows how closely the model's predictions align with true house prices on the test set. Points falling close to the diagonal reference line indicate accurate predictions.

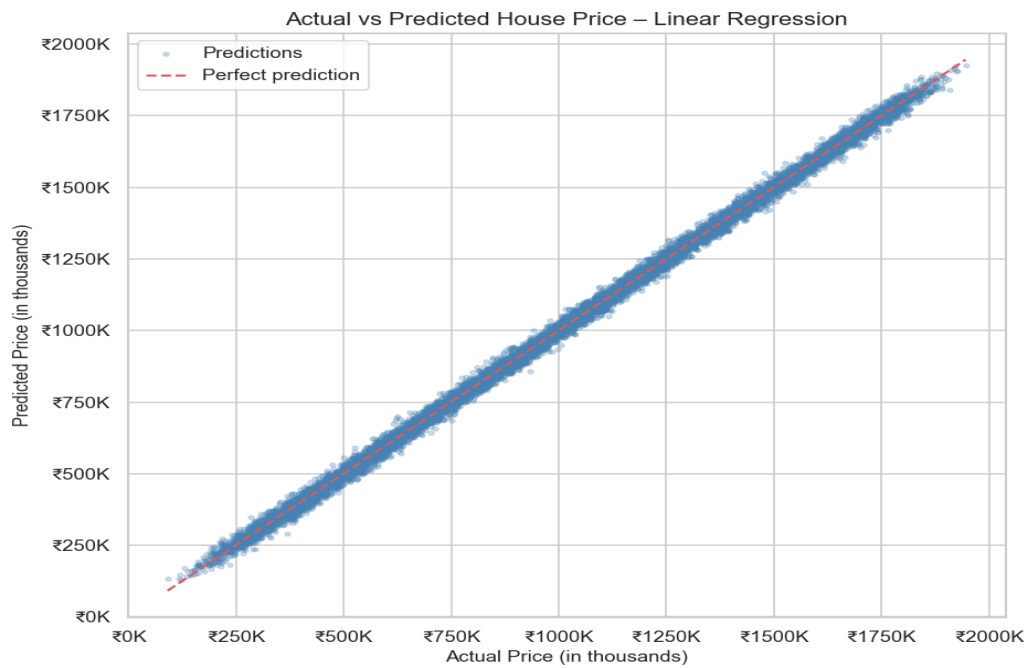


Figure 7: Actual vs Predicted House Price

The residual analysis confirms that prediction errors are approximately normally distributed around zero, with no significant systematic bias. This validates the appropriateness of the Linear Regression model for this dataset.

Residual Analysis – Linear Regression

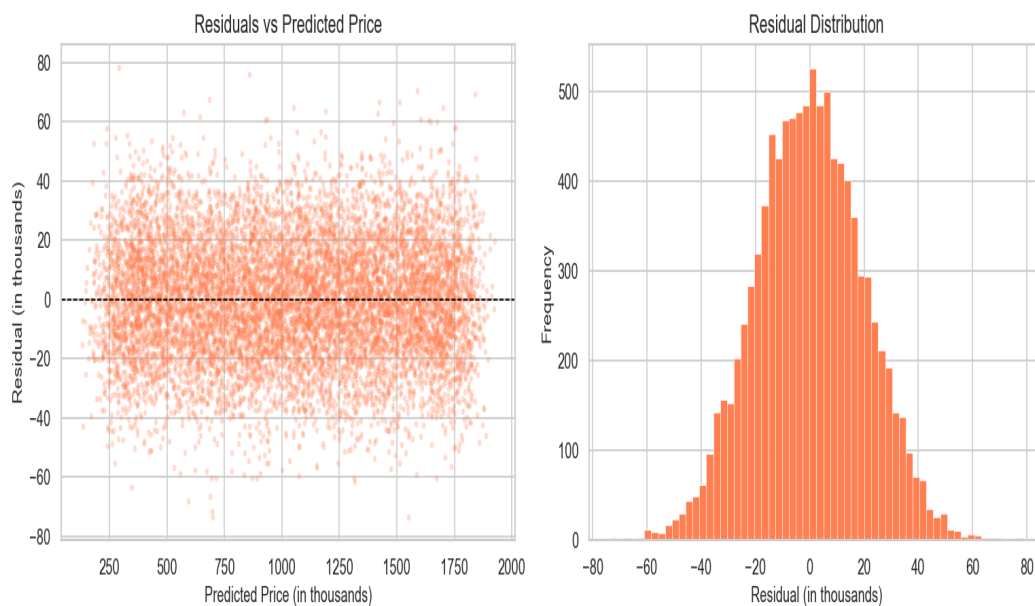


Figure 8: Residual Analysis

15. Sample Prediction

The final model is used to predict the price of a house with the following characteristics:

Feature	Value
area	2,000 sq ft
bedrooms	3
bathrooms	2
floors	2
age	10 years
distance	5 km
garage	Yes (1)
parking	Yes (1)
garden	Yes (1)
security	Yes (1)
school_nearby	Yes (1)
hospital_nearby	Yes (1)
shopping_mall_nearby	Yes (1)
public_transport	Yes (1)
crime_rate	0.2
population_density	5,000
location	medium
income_level	mid

Predicted House Price: 818,999.68

16. Key Findings

- Area is the strongest predictor of house price ($r = 0.991$, importance score 454,027).
- Location category (premium vs low) adds significant price value.
- House age is the third most influential feature — older properties are worth less.
- Income level of the neighbourhood is the fifth most important factor.
- Linear Regression outperforms all tree-based models on this dataset.
- All four models achieve $R^2 > 0.99$, confirming the dataset is well-structured.
- No missing values or duplicates were found in the 50,000-row dataset.

17. Limitations

- The dataset may be synthetically generated; real-world data typically contains more noise, missing values, and complex non-linear patterns.
- No cross-validation was applied — a single 80/20 split was used.

- No hyperparameter tuning was performed.
- Amenity features (school_nearby, hospital_nearby, etc.) showed near-zero correlation with price in this dataset.
- The model is trained on this specific dataset and may not generalise to real estate markets in different cities or countries.

18. Future Improvements

- Apply K-Fold cross-validation for more robust model performance estimates.
- Perform hyperparameter tuning using GridSearchCV or RandomizedSearchCV.
- Explore XGBoost and LightGBM for potentially better performance.
- Apply SHAP values for model interpretability and explainability.
- Deploy as a web application using Flask or Streamlit.
- Collect real-world housing data for practical predictions.
- Perform feature engineering — e.g., interaction terms, polynomial features.

19. Conclusion

This project successfully demonstrates a complete machine learning pipeline for house price prediction. Starting from raw tabular data with 50,000 records and 18 input features, we performed thorough data exploration, preprocessing, and trained four regression models using best practices to avoid data leakage.

The final selected model — Linear Regression — achieves an R^2 of 0.9981 and an RMSE of 19,941 on the held-out test set, demonstrating strong predictive power. Feature analysis reveals that property area is the dominant price driver, accounting for the overwhelming majority of predictive signal.

The project is fully reproducible, professionally structured, and ready for both academic submission and GitHub publication.

20. Technologies Used

Technology	Version	Purpose
Python	3.8+	Programming language
pandas	1.5+	Data loading, manipulation, EDA
numpy	1.23+	Numerical computation
matplotlib	3.6+	Data visualisation
seaborn	0.12+	Statistical visualisation
scikit-learn	1.2+	ML models, pipeline, preprocessing, metrics
joblib	1.2+	Model serialisation and loading
reportlab	3.x	PDF report generation

End of Report

House Price Prediction Using Machine Learning — B.Tech Computer Science Academic Project